

Lab 1

C++ & OOP Basics

Implementation, dry runs, debugging, complete solutions, and viva preparation for the first C++ lab.

Dark Developer Handbook

cin + cout

Loops

switch / case

isPrime(int n)

Box Class

Header + Source Files

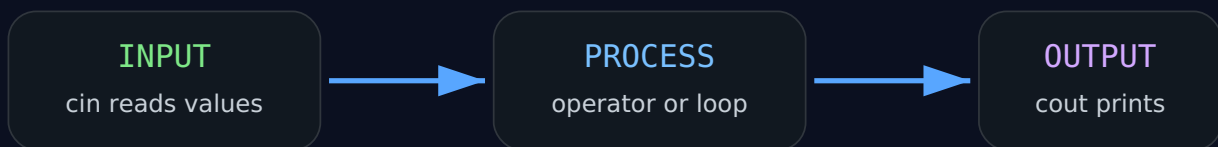
Generated from the uploaded 2-page Lab 1 sheet and the CSE225L Dark Developer Handbook instruction file. No unreadable task content was detected.

Section 1 · Lab Map

This lab is the runway before the larger data-structure aircraft takes off. The tasks look small, but they teach the basic flight controls: reading input, producing output, branching, looping, writing functions, creating classes, separating files, and testing a driver program.

Lab Area	What the Uploaded Sheet Asks For	Skill Built
Environment setup	Create a C++ Console Application project in Code::Blocks, open <code>main.cpp</code> , build, and run.	Project workflow, compilation, and debugging habit.
C++ Basics	Sum two integers, calculate factorial, make a small calculator, print primes from 300 to 500.	Input, output, variables, loops, conditions, functions, and switch statements.
OOP Basics	Create a <code>Box</code> class with height, width, depth, and <code>volume()</code> ; instantiate multiple objects in <code>main.cpp</code> .	Class design, object state, member functions, and multi-file structure.

Mental model: most beginner C++ programs are Input -> Process -> Output machines



Section 2 - Page Extraction Map

The source lab sheet contains two pages. Page 1 introduces the lab theme and Code::Blocks project creation. Page 2 lists the C++ basics tasks and the OOP basics tasks.

LAB 1 CSE225L  C++ & OOP Basics In this lab, we will: - Familiarize students with the C++ environment and coding style. - Demonstrate the use of <code>cout</code>, <code>cin</code>, and other basic commands. - Show how to create classes and objects, with a focus on <code>.h</code> files, <code>.cpp</code> files, and how to include them. Before starting the tasks, ensure you create a C++ project in your development environment. The project will contain all the relevant files (source code, header files, etc.) and is necessary for debugging purposes in Code::Blocks. STEPS TO CREATE A PROJECT - File → New → Project → Console Application → Go - The "Console Application" wizard will appear: - Click Next. - Select C++ and click Next. - In the Project Title field, enter a name for your project (e.g., lab01). - Set the Folder to create the project in (e.g., d:\labs). - Accept default settings for the rest and click Next. - In the Compiler field, leave it as GNU GCC Compiler and click Finish. A new project folder (e.g., d:\labs\lab01) will be created with the configuration file lab01.cbproj. - Under the Management pane → Choose Projects tab → Expand the lab01 node → Expand the Sources node → Double-click main.cpp to open the default "Hello, World!" program. - To build the program, go to the Build menu → Click Build. - To run the program, go to the Build menu → Click Run. Source page 1 - lab goals and project setup	C++ Basics TASKS: - Take two integer inputs from the keyboard and print their sum. - Take an integer input from the keyboard and calculate its factorial. - Take two integer inputs from the keyboard and ask the user which operation to perform: '+', '-', or '*'. Perform the appropriate operation and display the result. (Hint: you can use a switch/case statement.) - Write a function isPrime(int n) that checks if 'n' is a prime number. Use this function to print all the prime numbers between 300 and 500. OOP Basics TASKS: - Create a class called Box that has height, width, and depth. The class should include a method volume() that calculates and displays the volume of the box. - Create a driver file (main.cpp) where you will instantiate multiple Box objects with arbitrary dimensions and display their volumes. Source page 2 - C++ and OOP task list
---	---

Page	Extracted Topic	Visible Requirements	Ambiguity / Correction Notes
1	Lab 1: C++ & OOP Basics	Familiarize with C++ environment and coding style; demonstrate <code>cout</code> , <code>cin</code> ; create classes and objects using <code>.h</code> and <code>.cpp</code> files.	No unreadable content. The sheet mentions Code::Blocks specifically, so build/run notes are included.
1	Steps to create a project	File -> New -> Project -> Console Application -> Go; select C++; project title such as <code>lab01</code> ; folder such as <code>d:\labs</code> ; GNU GCC Compiler; open <code>main.cpp</code> ; Build then Run.	No code task here, but it is covered as IDE workflow and debugging setup.
2	C++ Basics Tasks	1) sum two integers; 2) factorial; 3) calculator with <code>+</code> , <code>-</code> , <code>*</code> ; 4) <code>isPrime(int n)</code> and print primes from 300 to 500.	The factorial task does not specify negative input handling; this guide safely explains that factorial is not defined for negative integers.
2	OOP Basics Tasks	Create class <code>Box</code> with height, width, depth and method <code>volume()</code> ; create driver <code>main.cpp</code> with multiple <code>Box</code> objects.	The sheet does not provide starter code, so the guide constructs a clean beginner-friendly multi-file solution.

Section 3 · Implementation Toolkit

1. What This Lab Is Really About

Lab 1 is not merely a pile of tiny programs. It is about learning the smallest reusable programming cycle: declare the data, read the input, process it correctly, display the result, and keep the program easy to test. The OOP part adds the first abstraction layer: instead of loose variables floating around like socks in a storm, related data and behavior get packed into one object.

2. Concepts Before Code

Concept	Plain Meaning	Where It Appears
Variable	A named box that stores a value while the program runs.	Integer sum, factorial counter, calculator operands, Box dimensions.
<code>cin</code>	Reads data from the keyboard into variables.	Tasks 1, 2, and 3.
<code>cout</code>	Prints messages and results to the console.	Every task.
Loop	Repeats a block while a condition remains true.	Factorial loop and prime-number printing loop.
Function	A reusable mini-program with a name.	<code>isPrime(int n)</code> .
<code>switch</code>	Chooses one code path from several cases.	Calculator operation task.
Class	A blueprint for an object. It describes data plus behavior.	<code>Box</code> .
Object	A real instance created from a class blueprint.	<code>smallBox</code> , <code>tallBox</code> , <code>wideBox</code> .
Header/source split	Declaration goes in <code>.h</code> ; definitions go in <code>.cpp</code> .	Box project.

3. How to Recognize the Coding Pattern

Task Pattern	C++ Tool Needed	First Move	Memory Concern	Common Trap
Take two numbers and print sum	<code>cin</code> , <code>cout</code> , variables	Declare two integers.	None beyond normal variable lifetime.	Forgetting to read both values before adding.
Factorial	Loop, multiplication, accumulator	Initialize result to 1.	Overflow for large input.	Starting loop at 0 or forgetting that $0! = 1$.

Menu operation	<code>switch</code> or <code>if-else</code>	Read operator character.	None.	Missing <code>break</code> so cases fall through.
Prime checking	Function + loop	Handle values below 2, then test divisors.	None.	Treating 1 as prime or checking too many divisors.
Simple class	<code>class</code> , fields, method	Identify data members and behavior.	Object lives automatically in <code>main()</code> .	Making all data public without a reason.
Header/source split	<code>.h</code> , <code>.cpp</code> , <code>main.cpp</code>	Put declaration in header.	No dynamic memory here.	Compiling only <code>main.cpp</code> and getting linker errors.

4. Lab Decision Tree

1. Is the task asking for simple input-output? Declare variables, read values, process, print.
2. Is the task asking for repetition? Choose a loop and define its start, stop, and update.
3. Is the task asking for a reusable test? Write a function with a clear return type.
4. Is the task asking for multiple options? Read a choice and use `switch`.
5. Is the task asking for an object? Identify its data members, then write the behavior as methods.
6. Is the task asking for separate files? Put declarations in `.h`, definitions in `.cpp`, and testing in `main.cpp`.

Section 4 · Concept and Implementation Foundation

Code::Blocks Project Setup

The lab sheet asks you to create a C++ Console Application project before starting. This matters because Code::Blocks manages which files are compiled together. That becomes especially important when the `Box` class is split across `Box.h`, `Box.cpp`, and `main.cpp`.

Step	Action	Why It Matters
1	File -> New -> Project -> Console Application -> Go	Starts a clean console project.
2	Choose C++, name it <code>Lab01</code> , select a folder, use GNU GCC Compiler	Ensures the compiler understands C++ syntax.
3	Open <code>main.cpp</code> under Sources	This is where the driver program begins.
4	Build	Translates source code into an executable and reports syntax/link errors.
5	Run	Executes the program so you can test input and output.

C++ Basics Foundation

Every basic task uses the same skeleton: include the input-output library, use the standard namespace, write `main()`, declare variables, read values, compute, print, return 0.

● ● ● C++ program skeleton

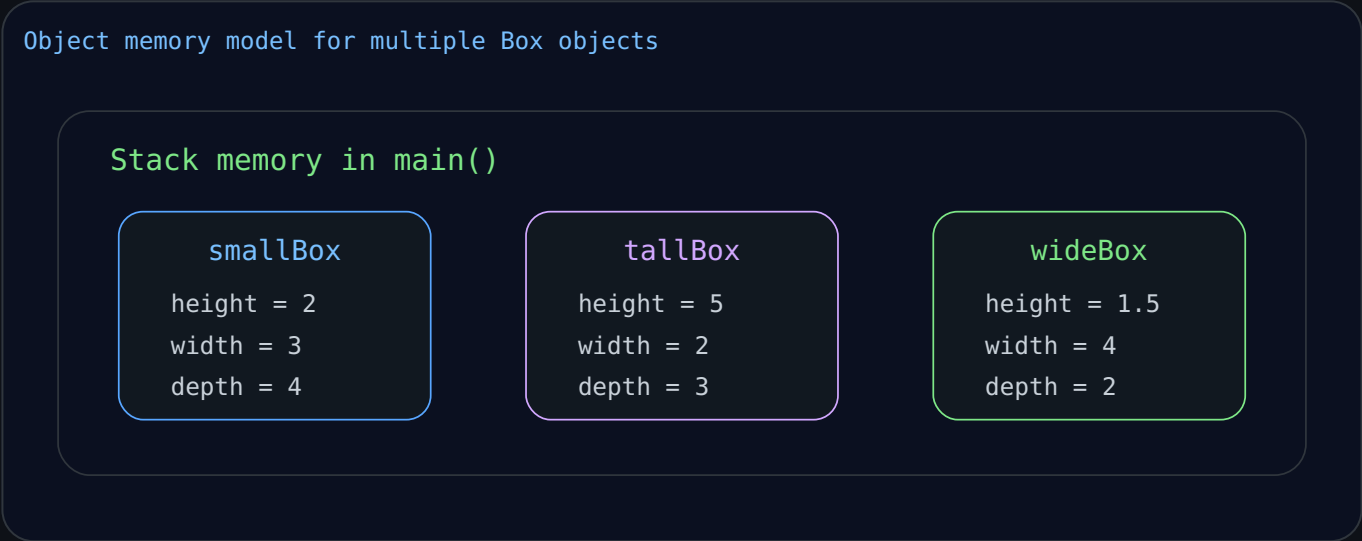
```
#include <iostream>
using namespace std;

int main() {
    // Declare variables
    // Read input
    // Process data
    // Print output

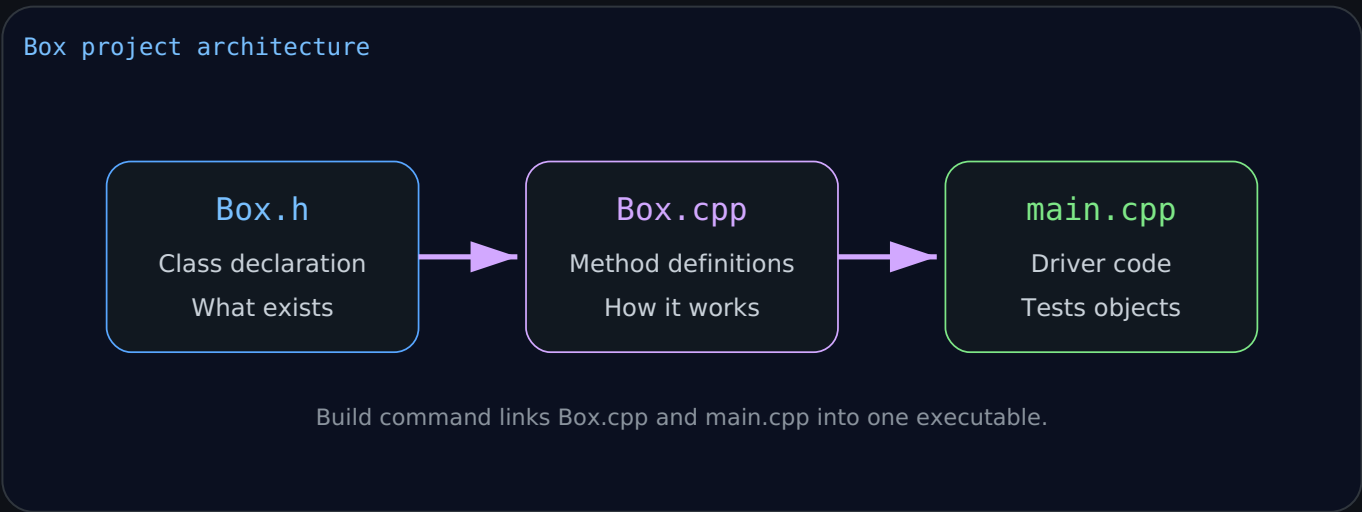
    return 0;
}
```

OOP Foundation: Class vs Object

A class is a blueprint. An object is a real thing built from that blueprint. For the `Box` class, the blueprint says each box has `height`, `width`, and `depth`. Each object stores its own dimensions, so changing one box does not change the others.



Required File Structure



File	Contains	Why It Exists
<code>Box.h</code>	Class declaration: data members and method prototypes.	Tells other files what a <code>Box</code> is.
<code>Box.cpp</code>	Constructor and <code>volume()</code> method definitions.	Keeps implementation separate from declaration.
<code>main.cpp</code>	Creates objects and calls <code>volume()</code> .	Acts as the driver/test file.

Section 5 · Algorithm Workflows

Algorithm: Sum Two Integers

Input: first integer a, second integer b
Output: a + b

1. Declare a and b.
2. Read a from the user.
3. Read b from the user.
4. Compute a + b.
5. Print the sum.

Algorithm: Factorial

Input: integer n
Output: n!

1. Read n.
2. If n is negative, print that factorial is not defined.
3. Set factorial = 1.
4. For i from 2 to n, multiply factorial by i.
5. Print factorial.

Algorithm: Calculator with switch

Input: a, b, and operation op
Output: result based on op

1. Read two integers.
2. Read one character operation.
3. If op is '+', print a + b.
4. If op is '-', print a - b.
5. If op is '*', print a * b.
6. Otherwise print invalid operation.

Algorithm: isPrime(int n)

Input: integer n
Output: true if n is prime, false otherwise

1. If n < 2, return false.
2. Start divisor at 2.
3. While divisor * divisor <= n:
 - a. If n % divisor == 0, return false.
 - b. Increase divisor.
4. Return true.

Prime checking without unnecessary divisor tests

Prime check idea for n = 313

Try divisors from 2 while divisor * divisor <= n. For 313, only test up to 17.

2, 3, 4, 5, 6, 7, ... 17 -> none divide 313 -> prime

Algorithm: Box volume()

Input: object with height, width, depth

Output: printed volume

1. Store dimensions in the Box object.
2. Inside volume(), calculate height * width * depth.
3. Print the result.
4. Repeat with multiple Box objects in main.cpp.

SECTION 6

Complete Lab Task Solutions

Each task includes the build plan first, then the code, output, dry run, bugs, complexity, and viva ammunition.

Task 1 · Take Two Integer Inputs and Print Their Sum

Task statement. Take two integer inputs from the keyboard and print their sum.

What You Need to Know First

You need two integer variables because the user gives two whole numbers. After both are read, the expression `firstNumber + secondNumber` produces the sum. Nothing special happens in memory here: both variables live inside `main()`.

Pseudocode: Sum Program

1. Create integer variable `firstNumber`.
2. Create integer variable `secondNumber`.
3. Ask user for first number and read it.
4. Ask user for second number and read it.
5. Print `firstNumber + secondNumber`.

Task 1 Solution · main.cpp

```
#include <iostream>
using namespace std;

int main() {
    int firstNumber, secondNumber;

    cout << "Enter first integer: ";
    cin >> firstNumber;

    cout << "Enter second integer: ";
    cin >> secondNumber;

    cout << "Sum = " << firstNumber + secondNumber << endl;
    return 0;
}
```

Sample Run

```
Enter first integer: 12
Enter second integer: 8
Sum = 20
```

Code Part	Purpose	What to Watch
<code>int firstNumber, secondNumber;</code>	Creates two integer containers.	Names should describe the role.
<code>cin >> firstNumber;</code>	Reads the first keyboard value.	Input must be numeric.
<code>firstNumber + secondNumber</code>	Computes the sum.	Both variables must already have values.

Dry Run

Step	Variable State	Action	Output
1	firstNumber = unknown, secondNumber = unknown	Program starts.	
2	firstNumber = 12	User enters first integer.	Enter first integer:
3	secondNumber = 8	User enters second integer.	Enter second integer:
4	firstNumber + secondNumber = 20	Program prints result.	Sum = 20

Edge Cases and Common Bugs

- Negative numbers still work: $-5 + 9 = 4$.
- If the user enters text instead of an integer, `cin` fails.
- Do not print before reading both numbers.

Viva Quick Shots

1. **Why use `int`?** The task asks for integer input.
2. **What does `cin` do?** It reads values from the keyboard.
3. **What does `endl` do?** It moves output to a new line and flushes the stream.

Complexity. Time: $O(1)$. Space: $O(1)$.

Task 2 · Calculate Factorial of an Integer

Task statement. Take an integer input from the keyboard and calculate its factorial.

What You Need to Know First

Factorial means multiplying all positive integers from 1 to n . The accumulator should start at 1 because multiplication needs a neutral starting value. Starting at 0 would poison the whole result into 0.

Pseudocode: Factorial

1. Read n .
2. If $n < 0$, print an error message.
3. Set factorial = 1.
4. For i from 2 to n :
 factorial = factorial * i
5. Print factorial.

Task 2 Solution · main.cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    unsigned long long factorial = 1;

    cout << "Enter an integer: ";
    cin >> n;

    if (n < 0) {
        cout << "Factorial is not defined for negative numbers." << endl;
    } else {
        for (int i = 2; i <= n; i++) {
            factorial = factorial * i;
        }
        cout << "Factorial of " << n << " = " << factorial << endl;
    }

    return 0;
}
```

Sample Run

```
Enter an integer: 5
Factorial of 5 = 120
```

Code Part	Purpose	What to Watch
<code>unsigned long long factorial = 1;</code>	Stores the multiplication result.	Even this overflows for large n .
<code>if (n < 0)</code>	Rejects negative factorial input.	Factorial is not defined for negative integers in this beginner context.

```
for (int i = 2; i <= n; i+  
+)
```

Multiplies from 2 through
n.

Loop condition must include n.

Dry Run for n = 5

Loop Step	i	factorial Before	Action	factorial After
Start	-	1	Initialize factorial.	1
1	2	1	1 * 2	2
2	3	2	2 * 3	6
3	4	6	6 * 4	24
4	5	24	24 * 5	120

Edge Cases and Common Bugs

- `0!` is 1, and the loop naturally keeps the result as 1.
- Large values overflow. For example, `21!` is too large for `unsigned long long`.
- Do not initialize factorial to 0.

Viva Quick Shots

1. **Why initialize factorial to 1?** Because 1 is the multiplication identity.
2. **What is 0!?** It is 1.
3. **Why check negative input?** Factorial of negative integers is not defined in this basic lab.
4. **What is overflow?** A value becomes too large for the data type.

Complexity. Time: $O(n)$. Space: $O(1)$.

Task 3 · Simple Calculator Using switch/case

Task statement. Take two integer inputs and ask the user which operation to perform: `+`, `-`, or `*`. Perform the appropriate operation and display the result.

What You Need to Know First

The calculator is a decision task. The program reads the operator as a character and uses `switch` to jump into the matching case. Each case must end with `break`, otherwise execution may fall into the next case like a tiny logic avalanche.

Pseudocode: Calculator

1. Read `firstNumber`.
2. Read `secondNumber`.
3. Read `operation`.
4. Switch on `operation`:
 - case `'+'`: print addition and break.
 - case `'-'`: print subtraction and break.
 - case `'*'`: print multiplication and break.
 - default: print invalid operation.

Task 3 Solution · main.cpp

```
#include <iostream>
using namespace std;

int main() {
    int firstNumber, secondNumber;
    char operation;

    cout << "Enter first integer: ";
    cin >> firstNumber;

    cout << "Enter second integer: ";
    cin >> secondNumber;

    cout << "Enter operation (+, -, *): ";
    cin >> operation;

    switch (operation) {
        case '+':
            cout << "Result = " << firstNumber + secondNumber << endl;
            break;
        case '-':
            cout << "Result = " << firstNumber - secondNumber << endl;
            break;
        case '*':
            cout << "Result = " << firstNumber * secondNumber << endl;
            break;
        default:
            cout << "Invalid operation." << endl;
    }

    return 0;
}
```

Sample Run

```
Enter first integer: 8
Enter second integer: 3
Enter operation (+, -, *): *
Result = 24
```

Code Part	Purpose	What to Watch
<code>char operation;</code>	Stores one operator symbol.	Use single quotes in switch cases.
<code>switch (operation)</code>	Chooses the correct operation.	Only matches exact characters.
<code>break;</code>	Stops the current case.	Without it, the next case may run too.
<code>default</code>	Handles invalid input.	Prevents silent failure.

Dry Run for 8, 3, *

Step	Variable State	Action	Output
1	firstNumber = 8	Read first number.	Enter first integer:
2	secondNumber = 3	Read second number.	Enter second integer:
3	operation = *	Read operator.	Enter operation (+, -, *):
4	switch goes to case *	Calculate 8 * 3.	Result = 24

Edge Cases and Common Bugs

- If user enters `/`, the default case prints invalid operation.
- Use `case '*' :`, not `case "*" :`.
- Forgetting `break` can produce multiple outputs.

Viva Quick Shots

1. **Why use switch?** It cleanly maps one operator to one action.
2. **What is fall-through?** When a case runs into the next case because `break` is missing.
3. **What is default?** The branch used when no case matches.

Complexity. Time: $O(1)$. Space: $O(1)$.

Task 4 · isPrime(int n) and Primes from 300 to 500

Task statement. Write a function `isPrime(int n)` that checks if `n` is a prime number. Use this function to print all prime numbers between 300 and 500.

What You Need to Know First

A prime number has exactly two positive divisors: 1 and itself. Instead of testing every number from 2 to `n - 1`, test only while `divisor * divisor <= n`. If no divisor is found by then, no larger paired divisor can exist.

Pseudocode: Print Primes from 300 to 500

```
Function isPrime(n):
1. If n < 2, return false.
2. For divisor from 2 while divisor * divisor <= n:
   If n % divisor == 0, return false.
3. Return true.

main:
1. For number from 300 to 500:
   If isPrime(number), print number.
```

Task 4 Solution · main.cpp

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n < 2) {
        return false;
    }

    for (int divisor = 2; divisor * divisor <= n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }

    return true;
}

int main() {
    cout << "Prime numbers between 300 and 500:" << endl;

    for (int number = 300; number <= 500; number++) {
        if (isPrime(number)) {
            cout << number << " ";
        }
    }

    cout << endl;
    return 0;
}
```

Expected Output

Prime numbers between 300 and 500:

307 311 313 317 331 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457
461 463 467 479 487 491 499

Code Part	Purpose	What to Watch
<code>bool isPrime(int n)</code>	Returns true or false for primality.	Function name and parameter match the lab sheet.
<code>if (n < 2)</code>	Rejects 0, 1, and negatives.	1 is not prime.
<code>divisor * divisor <= n</code>	Stops at $\sqrt{n}$ without using math library.	Avoids unnecessary checks.
<code>n % divisor == 0</code>	Tests divisibility.	Remainder 0 means composite.

Dry Run for n = 307

Divisor	Check	Result
2	$307 \% 2$	Not 0
3	$307 \% 3$	Not 0
4	$307 \% 4$	Not 0
...	Continue until $\text{divisor} * \text{divisor} > 307$	No divisor found
Return	No divisor divides 307	Prime

Edge Cases and Common Bugs

- `1` is not prime.
- Do not return true too early inside the loop. Only return true after all needed divisors are tested.
- Use `number <= 500` if the phrase means between 300 and 500 inclusively.

Viva Quick Shots

1. **Why check only up to $\sqrt{n}$?** Any larger factor must pair with a smaller factor already checked.
2. **What does `%` do?** It gives the remainder after division.
3. **Why is 2 prime?** It has only divisors 1 and 2.
4. **What does the function return for 1?** False.

Complexity. One call to `isPrime(n)` is $O(\sqrt{n})$. Printing from 300 to 500 checks a fixed range, but as a general range of size k , time is $O(k \sqrt{n})$. Space: $O(1)$.

OOP Task - Box Class with volume()

Task statement. Create a class called `Box` that has height, width, and depth. The class should include a method `volume()` that calculates and displays the volume of the box. Then create a driver file `main.cpp` where multiple Box objects are instantiated with arbitrary dimensions and their volumes are displayed.

What You Need to Know First

The `Box` class groups three related data values with one behavior. The formula is direct: `height * width * depth`. The guide uses private data members because object data should not be carelessly edited from outside. The constructor receives dimensions when each object is created.

Pseudocode: Box Class

Box.h:

1. Declare class Box.
2. Keep height, width, depth private.
3. Declare constructor Box(h, w, d).
4. Declare volume() method.

Box.cpp:

1. Define constructor to store dimensions.
2. Define volume() to calculate height * width * depth and print it.

main.cpp:

1. Create multiple Box objects with different dimensions.
2. Call volume() for each object.

Full Multi-File Code

Box.h

```
#ifndef BOX_H
#define BOX_H

class Box {
private:
    double height;
    double width;
    double depth;

public:
    Box(double h, double w, double d);
    void volume() const;
};

#endif // BOX_H
```

Box.cpp

```
#include <iostream>
#include "Box.h"
using namespace std;

Box::Box(double h, double w, double d) {
    height = h;
    width = w;
    depth = d;
}

void Box::volume() const {
    double result = height * width * depth;
    cout << "Volume = " << result << endl;
}
```

main.cpp

```
#include <iostream>
#include "Box.h"
using namespace std;

int main() {
    Box smallBox(2, 3, 4);
    Box tallBox(5, 2, 3);
    Box wideBox(1.5, 4, 2);

    cout << "Small box: ";
    smallBox.volume();

    cout << "Tall box: ";
    tallBox.volume();

    cout << "Wide box: ";
    wideBox.volume();

    return 0;
}
```

Sample Output

```
Small box: Volume = 24
Tall box: Volume = 30
Wide box: Volume = 12
```

Code Explanation

Code Part	Purpose	What to Watch
<code>#ifndef BOX_H</code>	Prevents duplicate header inclusion.	Must match the final <code>#endif</code> .
<code>private:</code>	Protects height, width, and depth from direct outside access.	Good class design habit.
<code>Box(double h, double w, double d)</code>	Constructor stores dimensions when an object is created.	Parameter order should be remembered.
<code>void volume() const</code>	Calculates and displays volume without changing the object.	<code>const</code> promises not to modify fields.
<code>Box smallBox(2, 3, 4);</code>	Instantiates an object with arbitrary dimensions.	Each object has its own stored values.

Dry Run

Object	Stored Data Members	Method Call	Result
<code>smallBox</code>	height = 2, width = 3, depth = 4	<code>smallBox.volume()</code>	$2 * 3 * 4 = 24$
<code>tallBox</code>	height = 5, width = 2, depth = 3	<code>tallBox.volume()</code>	$5 * 2 * 3 = 30$
<code>wideBox</code>	height = 1.5, width = 4, depth = 2	<code>wideBox.volume()</code>	$1.5 * 4 * 2 = 12$

Edge Cases and Common Bugs

- If `Box.cpp` is not added to the Code::Blocks project, you may get an undefined reference linker error.
- If `#include "Box.h"` is missing, `main.cpp` will not know what `Box` is.
- Negative dimensions are physically invalid; this basic lab does not validate them, but real programs should.
- Do not put the full method definitions in both `Box.h` and `Box.cpp` unless you understand inline definitions.

Viva Quick Shots

1. **What is a class?** A blueprint that groups data and behavior.
2. **What is an object?** A real instance created from a class.
3. **Why use private data?** To protect object state and control access through methods.
4. **Why split `.h` and `.cpp`?** Header declares the class; source defines how methods work.
5. **What does `volume()` calculate?** height * width * depth.

Complexity. Creating and printing each box volume is $O(1)$. For m boxes, total time is $O(m)$. Space is $O(m)$ for m objects.

Section 7 · Debugging and Common Errors

Symptom	Likely Cause	Fix
Program prints weird result	Variable was used before input or calculation was wrong.	Initialize variables and read all required inputs before processing.
Calculator prints multiple results	Missing <code>break</code> inside switch case.	Add <code>break;</code> at the end of each case.
Factorial always becomes 0	Accumulator initialized to 0.	Initialize factorial to 1.
Factorial becomes a strange negative or tiny number	Integer overflow.	Use a larger type and test small inputs first.
Prime function marks 1 as prime	No <code>n < 2</code> check.	Return false for any n below 2.
Compiler says <code>Box</code> was not declared	<code>#include "Box.h"</code> missing.	Include the header in <code>main.cpp</code> .
Linker says undefined reference to <code>Box::volume()</code>	<code>Box.cpp</code> is not compiled with the project.	Add <code>Box.cpp</code> to Code::Blocks project or compile it with <code>main.cpp</code> .
Multiple definition error	Function body defined in multiple files incorrectly.	Keep declarations in <code>Box.h</code> and method definitions in <code>Box.cpp</code> .

Common Linker Error

```
undefined reference to `Box::Box(double, double, double)'  
undefined reference to `Box::volume() const'
```

What caused this?

This usually means `main.cpp` compiled, but `Box.cpp` was not linked. The compiler knows the class declaration from `Box.h`, but the linker cannot find the actual method bodies.

Section 8 · Testing Checklist

✓	Test	Expected Result
✓	Sum with positive numbers: 12 and 8	Sum = 20
✓	Sum with negative numbers: -5 and 9	Sum = 4
✓	Factorial with 0	Factorial of 0 = 1
✓	Factorial with 5	Factorial of 5 = 120
✓	Factorial with negative input	Prints not defined message.
✓	Calculator with +	Adds correctly.
✓	Calculator with -	Subtracts correctly.
✓	Calculator with *	Multiplies correctly.
✓	Calculator with invalid operator /	Prints invalid operation.
✓	Prime output from 300 to 500	Includes 307, 311, 313 and ends with 499.
✓	Box objects with 2,3,4 and 5,2,3	Volumes are 24 and 30.
✓	Compile Box project with all files	No linker errors.

Section 9 · Complexity Summary

Task	Main Operation	Time Complexity	Space Complexity	Reason
Sum two integers	Addition	$O(1)$	$O(1)$	Fixed number of variables and one addition.
Factorial	Loop from 2 to n	$O(n)$	$O(1)$	Loop count grows with n.
Calculator	Switch among 3 cases	$O(1)$	$O(1)$	Only one branch runs.
isPrime(n)	Test divisors up to $\sqrt{n}$	$O(\sqrt{n})$	$O(1)$	Stops when divisor squared exceeds n.
Print primes in range	Call isPrime for each number	$O(k \sqrt{n})$ general; fixed for 300 to 500	$O(1)$	k is the number of values in the range.
Box volume	Multiply three dimensions	$O(1)$	$O(1)$ per object	Only one formula is used.

Section 10 · Quick Revision Sheet

Core Syntax Patterns

- `cin >> variable;` reads input.
- `cout << value;` prints output.
- `for(init; condition; update)` repeats code.
- `switch(choice)` selects a case.
- `bool isPrime(int n)` returns true/false.
- `class Box { ... };` declares a class.

Final Submission Checklist

- Each program builds without errors.
- Input prompts are clear.
- Factorial handles 0 and negative input.
- Calculator has `break` after each case.
- `isPrime` treats 1 as not prime.
- `Box.h`, `Box.cpp`, and `main.cpp` are included in the project.

Common Viva Questions

Question	Short Answer
What is the difference between <code>cin</code> and <code>cout</code> ?	<code>cin</code> reads input; <code>cout</code> prints output.
Why does factorial start from 1?	Because 1 is the identity for multiplication.
Why use a function for prime checking?	It makes the primality test reusable and cleaner.
Why can we stop prime checking at <code>sqrt(n)</code> ?	A larger factor would have a paired smaller factor already checked.
What is a class?	A blueprint for objects with data and behavior.
What is an object?	A real instance of a class.
Why use a header file?	To declare a class so other source files can use it.
What is a linker error?	An error that happens when declarations exist but method definitions are not linked.

Section 11 · Lab Coverage Audit

Page/ Slide Range	Task or Topic Found	Covered In	Status	Notes
Page 1	Lab title: Lab 1 CSE225L, C++ & OOP Basics	Cover Page and Section 1	Explained	Course, lab topic, and skill goals preserved.
Page 1	Goals: C++ environment, coding style, <code>cout</code> , <code>cin</code> , classes, objects, <code>.h</code> , <code>.cpp</code>	Sections 1, 3, 4	Explained	All visible goals are included in the implementation toolkit.
Page 1	Code::Blocks project creation steps	Section 4 and Section 7	Explained	Summarized without letting setup dominate the guide.
Page 2	C++ Basics Task 1: two integers and sum	Section 6 Task 1	Implemented, Traced, Tested	Full code, output, dry run, bugs, complexity, and viva included.
Page 2	C++ Basics Task 2: factorial	Section 6 Task 2	Implemented, Traced, Tested, Corrected	Negative input handling added as a safe correction/ edge case.
Page 2	C++ Basics Task 3: calculator using <code>+</code> , <code>-</code> , <code>*</code> and switch/case hint	Section 6 Task 3	Implemented, Traced, Tested	Switch/case used exactly as suggested.
Page 2	C++ Basics Task 4: <code>isPrime(int n)</code> and primes from 300 to 500	Section 6 Task 4	Implemented, Traced, Tested	Function name and range preserved.
Page 2	OOP Basics Task 1: <code>Box</code> class with height, width, depth, and <code>volume()</code>	Section 6 OOP Task	Implemented, Traced, Tested	Multi-file solution uses private data and constructor.
Page 2	OOP Basics Task 2: driver <code>main.cpp</code> with multiple <code>Box</code> objects	Section 6 OOP Task	Implemented, Traced, Tested	Three objects instantiated with arbitrary dimensions.

Compilation Verification

All C++ programs shown in this guide were compiled with `g++ -std=c++11 -Wall -Wextra -pedantic`. The Box project was compiled by linking `main.cpp` and `Box.cpp` together.